

SketchLang

A Mini Domain-Specific Language for Vector Graphics Generation

Compiler Design Laboratory Project — Phase 1: Problem Definition & Design

Student Name	Nandana Krishnan
Register No.	24BCE2081
Course	Compiler Design Laboratory
Project Type	Individual
Phase	Phase 1 of 3 — Problem Definition and System Design
Language / Tools	Python 3, regular-expression based lexer, recursive-descent parser, SVG

1. Project Title

SketchLang: A Mini Domain-Specific Language for Vector Graphics Generation

2. Abstract

SketchLang is a small, purpose-built programming language and compiler that translates simple, human-readable drawing instructions (for example “pen red”, “forward 100”, “turn 90”) into a scalable vector graphics (SVG) image. The project implements the complete front-to-back pipeline of a compiler — lexical analysis, syntax analysis, semantic analysis, symbol-table management, intermediate code generation, a basic optimization pass, and target code generation — around a domain that is visual and easy to demonstrate. Unlike a purely numeric or textual toy language, every successful or failed compilation is immediately visible as a picture, which makes the project simple to explain, simple to test, and easy to extend with original features in later phases.

3. Problem Statement

General-purpose drawing is usually done either by hand in a graphics editor or by writing verbose, general-purpose code against a graphics library. There is no lightweight way for a user to describe a picture procedurally, using a small set of intuitive commands, and have it compiled directly into a portable image format. The project addresses this gap by designing and implementing a minimal language, along with a compiler that validates, optimizes, and renders programs written in it.

4. Motivation

Turtle-graphics-style languages (e.g. Logo) are well known for teaching programming concepts, but they are usually interpreted directly rather than compiled. Building a real compiler pipeline — including an intermediate representation and an optimization pass — around a visual, turtle-graphics-like language keeps the project approachable while still exercising every major stage taught in the Compiler Design course. The visual output also makes every review and demo self-explanatory: correctness and optimization can literally be seen.

5. Objectives

- Design a small, well-specified language (tokens, keywords, grammar) for describing 2D line drawings.
- Implement a lexer that tokenizes SketchLang source files and reports lexical errors with line numbers.
- Implement a recursive-descent parser that builds an Abstract Syntax Tree (AST) and reports syntax errors.
- Implement semantic checks (valid colors, valid repeat counts, undefined variables in later phases).
- Generate an intermediate representation (IR) of drawing instructions, independent of the output format.
- Implement an optimization pass that merges redundant/collinear IR instructions.
- Generate SVG target code from the optimized IR and render it as the final image.

6. Scope

The initial scope covers straight-line 2D drawings driven by pen, movement, turning, and a bounded repeat/looping construct, with SVG as the sole target output. Curves, fill regions, user-defined procedures, and multiple output formats are treated as optional stretch goals for Phase 2/3 rather than Phase 1 requirements, keeping the initial system small enough to fully understand and defend individually.

7. Background Study

The design draws on two established ideas: (1) turtle graphics, popularized by the Logo programming language, where a virtual pen moves and turns to trace a path; and (2) the classical compiler pipeline (lexer → parser → semantic analysis → IR → optimization → code generation) as taught in standard compiler design texts. SVG was chosen as the target format because it is a simple, human-readable, standard vector format that any browser can render without extra tooling, which keeps the “target machine” for this project easy to reason about.

8. Compiler Design Concepts Involved

- Lexical Analysis — regular-expression-driven tokenizer producing a token stream with line tracking.
- Syntax Analysis / Parsing — hand-written recursive-descent parser implementing the SketchLang grammar.
- Abstract Syntax Tree — nested Node objects representing statements and the repeat block structure.
- Semantic Analysis — validation of colors and repeat counts; extensible to variable/type checks.
- Symbol Table Management — planned in Phase 2 to support variables (e.g. “let side = 100”).
- Intermediate Code Generation — a flat, three-address-code-like list of LINE/MOVE instructions.
- Code Optimization — a peephole-style pass merging consecutive collinear line segments.
- Target Code Generation — emission of valid SVG markup from the optimized IR.
- Error Detection — distinct Lexical, Syntax, and Semantic error classes with line-numbered messages.

9. Proposed Methodology

The compiler is implemented as a linear pipeline of independent, testable stages, mirroring the classical compiler architecture. Each stage consumes the output of the previous stage and can be tested in isolation, which supports incremental development across the three project phases.

9.1 Pipeline

Source (.sketch)	Lexer	Parser → AST	SemanticAn alyzer	IRGenerator	Optimizer	SVGCodegen	Image
---------------------	-------	-----------------	----------------------	-------------	-----------	------------	-------

10. System Architecture

The system follows a classic single-pass-front-end / multi-pass-back-end architecture. The front end (lexer, parser, semantic analyzer) validates the program and produces an AST. A tree-walking generator converts the AST into a flat IR using a small amount of interpreter-like state (current position, heading, and pen color) so that loops (“repeat”) are expanded into concrete drawing instructions. The optimizer then simplifies the IR, and the back end emits SVG.

10.1 Module Overview

- lexer.py (module) — tokenize(source) → List[Token]
- parser.py (module) — Parser.parse_program() → AST
- semantics.py (module) — check_semantics(ast) — raises SemanticError on failure
- ir.py (module) — generate_ir(ast) → List[IR instruction]
- optimizer.py (module) — optimize_ir(ir) → List[IR instruction] (fewer or equal length)
- codegen.py (module) — generate_svg(ir) → SVG string
- In the Phase 1 prototype all modules are combined into a single file, sketchlang.py, for simplicity; Phase 2 will split them into separate modules as listed above.

11. Technology Stack

Language	Python 3.12
Lexer	Custom, built on Python's re module (no external lexer generator)
Parser	Hand-written recursive-descent parser
Target format	SVG (Scalable Vector Graphics), rendered via any browser or image viewer
Dev tools	VS Code, Git for version control
Testing	Python's built-in unittest module (Phase 2)

12. Initial Prototype

A working command-line prototype has been implemented in sketchlang.py. It accepts a .sketch source file, runs it through the full pipeline, prints the unoptimized and optimized IR for inspection, and writes the resulting drawing to output.svg. Below is a sample program and the image it compiles to.

12.1 Sample SketchLang Program

```
# Draw a colorful pinwheel-ish pattern using a square + a star
pen blue
repeat 4 {
    forward 100
    turn 90
}

move 0 0
pen red
repeat 8 {
    forward 60
    turn 135
}

move -150 -150
pen green
forward 50
turn 90
forward 50
```

```
turn 90  
forward 50
```

12.2 Compiled Output

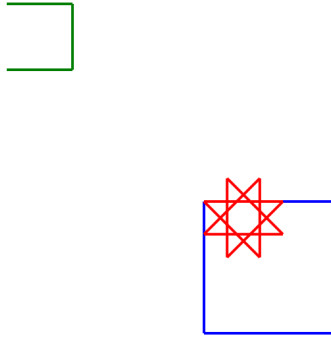


Figure 1: SVG image rendered from the sample program above — a blue square, a red eight-pointed star drawn with a repeat loop, and a green partial square, each in the color set by the preceding “pen” command.

12.3 Optimizer and Error Handling — Evidence

Running two consecutive forward commands in the same direction is merged by the optimizer into a single line instruction, reducing 3 IR instructions to 2:

```
pen purple  
forward 40  
forward 60  
turn 90  
forward 50  
  
> Unoptimized IR: 3 instructions  
> Optimized IR: 2 instructions (two collinear segments merged into one)
```

An invalid color is caught by the semantic analyzer before any code is generated:

```
pen skyblue  
forward 50  
  
> Compilation failed: Semantic error (line 1): unknown color 'skyblue'.  
> Valid colors: ['black', 'blue', 'brown', 'green', 'orange', 'pink', 'purple', 'red']
```

Appendix A: Phase 2 & Phase 3 Roadmap (for reference)

Phase 2 — Core Implementation (planned)

- Add variables (“let size = 100”) and a real symbol table (name → value, scope).
- Extend semantic analysis to catch undefined-variable and redeclaration errors.
- Split the single-file prototype into proper modules (lexer, parser, semantics, ir, optimizer, codegen).
- Add a second, independent optimization: dead-code elimination for movements that never draw.
- Add unit tests for each stage and a small suite of valid/invalid sample programs.
- Write the implementation progress report for Review 2.

Phase 3 — Final Implementation and Testing (planned)

- Add a second target backend (e.g. PNG raster export or a simple bytecode + interpreter) to demonstrate retargetability.
- Add boundary/error test cases: deeply nested repeats, malformed braces, huge coordinates.
- Measure and report compilation time and IR-size reduction from optimization across several sample programs.
- Finalize documentation, user instructions, and prepare the live demonstration for Review 3.

Appendix B: Phase 2 & Phase 3 Roadmap (for reference)

Prototype Code -> sketchlang.py

"""

SketchLang - a tiny Domain-Specific Language that compiles simple turtle-style drawing commands into an SVG image.

Grammar (EBNF):

```
program  := statement*
statement := pen_stmt | forward_stmt | turn_stmt | move_stmt | repeat_stmt
pen_stmt  := "pen" COLOR
forward_stmt := "forward" NUMBER
turn_stmt  := "turn" NUMBER
move_stmt  := "move" NUMBER NUMBER
repeat_stmt := "repeat" NUMBER "{" statement* "}"
```

Pipeline implemented below (Phase-1 prototype -- scaled up in Phase 2/3):

```
source text -> Lexer -> tokens -> Parser -> AST -> Semantic Analyzer
            -> IR generator -> Optimizer -> SVG Code Generator -> output.svg
```

"""

```
import math
```

```
import re
```

```
import sys
```

```
# -----
```

```
# 1. LEXICAL ANALYSIS
```

```
# -----
```

```
KEYWORDS = {"pen", "forward", "turn", "move", "repeat"}
```

```
COLORS = {"red", "green", "blue", "black", "orange", "purple", "brown", "pink"}
```

```
TOKEN_SPEC = [
```

```
    ("NUMBER", r"-?\d+(\.\d+)?"),
```

```
    ("LBRACE", r"\{"),
```

```
    ("RBRACE", r"\}"),
```

```
    ("IDENT", r"[A-Za-z_][A-Za-z0-9_]*"),
```

```
    ("SKIP", r"[\t]+"),
```

```
    ("NEWLINE", r"\n"),
```

```
    ("COMMENT", r"#.*"),
```

```
    ("MISMATCH", r"."),
```

```
]
MASTER_RE = re.compile("|".join(f"(?P<{n}>{p})" for n, p in TOKEN_SPEC))
```

```
class Token:
    def __init__(self, kind, value, line):
        self.kind, self.value, self.line = kind, value, line

    def __repr__(self):
        return f"Token({self.kind}, {self.value!r}, line={self.line})"
```

```
class LexError(Exception):
    pass
```

```
def tokenize(source):
    tokens = []
    line = 1
    for m in MASTER_RE.finditer(source):
        kind = m.lastgroup
        value = m.group()
        if kind == "NEWLINE":
            line += 1
            continue
        if kind in ("SKIP", "COMMENT"):
            continue
        if kind == "MISMATCH":
            raise LexError(f"Lexical error: unexpected character {value!r} on line {line}")
        if kind == "IDENT" and value in KEYWORDS:
            kind = value.upper()
        tokens.append(Token(kind, value, line))
    tokens.append(Token("EOF", None, line))
    return tokens
```

```
# -----
# 2. SYNTAX ANALYSIS (recursive-descent parser -> AST)
# -----
```

```
class ParseError(Exception):
```

```
pass
```

```
class Node:
```

```
    """Generic AST node: kind + args + line (for error messages)."""
```

```
    def __init__(self, kind, args, line):
```

```
        self.kind, self.args, self.line = kind, args, line
```

```
    def __repr__(self):
```

```
        return f"Node({self.kind}, {self.args})"
```

```
class Parser:
```

```
    def __init__(self, tokens):
```

```
        self.tokens = tokens
```

```
        self.pos = 0
```

```
    def peek(self):
```

```
        return self.tokens[self.pos]
```

```
    def advance(self):
```

```
        tok = self.tokens[self.pos]
```

```
        self.pos += 1
```

```
        return tok
```

```
    def expect(self, kind):
```

```
        tok = self.peek()
```

```
        if tok.kind != kind:
```

```
            raise ParseError(f"Syntax error: expected {kind} but got {tok.kind} on line {tok.line}")
```

```
        return self.advance()
```

```
    def parse_program(self):
```

```
        statements = []
```

```
        while self.peek().kind != "EOF":
```

```
            statements.append(self.parse_statement())
```

```
        return Node("PROGRAM", statements, 0)
```

```
    def parse_statement(self):
```

```
        tok = self.peek()
```

```
        if tok.kind == "PEN":
```

```

        self.advance()
        color_tok = self.expect("IDENT")
        return Node("PEN", [color_tok.value], tok.line)
    if tok.kind == "FORWARD":
        self.advance()
        n = self.expect("NUMBER")
        return Node("FORWARD", [float(n.value)], tok.line)
    if tok.kind == "TURN":
        self.advance()
        n = self.expect("NUMBER")
        return Node("TURN", [float(n.value)], tok.line)
    if tok.kind == "MOVE":
        self.advance()
        x = self.expect("NUMBER")
        y = self.expect("NUMBER")
        return Node("MOVE", [float(x.value), float(y.value)], tok.line)
    if tok.kind == "REPEAT":
        self.advance()
        n = self.expect("NUMBER")
        self.expect("LBRACE")
        body = []
        while self.peek().kind != "RBRACE":
            if self.peek().kind == "EOF":
                raise ParseError(f"Syntax error: missing '}' for repeat opened near line {tok.line}")
            body.append(self.parse_statement())
        self.expect("RBRACE")
        return Node("REPEAT", [int(float(n.value)), body], tok.line)
    raise ParseError(f"Syntax error: unexpected token {tok.kind} on line {tok.line}")

```

```

# -----
# 3. SEMANTIC ANALYSIS
# -----

```

```

class SemanticError(Exception):
    pass

```

```

def check_semantics(program_node):
    """Simple semantic checks: valid colors, positive repeat counts."""

```

```

def walk(statements):
    for stmt in statements:
        if stmt.kind == "PEN":
            color = stmt.args[0]
            if color not in COLORS:
                raise SemanticError(
                    f"Sematic error (line {stmt.line}): unknown color '{color}'. "
                    f"Valid colors: {sorted(COLORS)}"
                )
            elif stmt.kind == "REPEAT":
                count, body = stmt.args
                if count <= 0:
                    raise SemanticError(
                        f"Sematic error (line {stmt.line}): repeat count must be positive, got {count}"
                    )
                walk(body)
    walk(program_node.args)

```

```

# -----
# 4. INTERMEDIATE REPRESENTATION (a flat "three-address-code"-like list)
# -----
# Each IR instruction is a tuple:
# ("LINE", x1, y1, x2, y2, color) -- draw a segment
# ("MOVE", x, y) -- reposition pen without drawing

```

```

def generate_ir(program_node):
    ir = []
    state = {"x": 0.0, "y": 0.0, "heading": 0.0, "color": "black"}

    def exec_stmt(stmt):
        if stmt.kind == "PEN":
            state["color"] = stmt.args[0]
        elif stmt.kind == "FORWARD":
            dist = stmt.args[0]
            rad = math.radians(state["heading"])
            nx = state["x"] + dist * math.cos(rad)
            ny = state["y"] + dist * math.sin(rad)
            ir.append(("LINE", state["x"], state["y"], nx, ny, state["color"]))
            state["x"], state["y"] = nx, ny
        elif stmt.kind == "TURN":

```

```

        state["heading"] = (state["heading"] + stmt.args[0]) % 360
    elif stmt.kind == "MOVE":
        state["x"], state["y"] = stmt.args
        ir.append(("MOVE", state["x"], state["y"]))
    elif stmt.kind == "REPEAT":
        count, body = stmt.args
        for _ in range(count):
            for s in body:
                exec_stmt(s)

for stmt in program_node.args:
    exec_stmt(stmt)
return ir

```

```

# -----
# 5. OPTIMIZATION
# -----

```

```

# Merge consecutive LINE instructions that are collinear and share a color
# into a single instruction, reducing the number of emitted SVG elements.

```

```

def optimize_ir(ir):
    if not ir:
        return ir
    optimized = [ir[0]]
    for instr in ir[1:]:
        prev = optimized[-1]
        if (
            instr[0] == "LINE"
            and prev[0] == "LINE"
            and instr[5] == prev[5]      # same color
            and (prev[3], prev[4]) == (instr[1], instr[2]) # prev end == this start
            and _collinear(prev[1], prev[2], prev[3], prev[4], instr[3], instr[4])
        ):
            optimized[-1] = ("LINE", prev[1], prev[2], instr[3], instr[4], prev[5])
        else:
            optimized.append(instr)
    return optimized

```

```

def _collinear(x1, y1, x2, y2, x3, y3, eps=1e-6):

```

```

# cross product of (p2-p1) and (p3-p1) close to zero => collinear
cross = (x2 - x1) * (y3 - y1) - (y2 - y1) * (x3 - x1)
return abs(cross) < eps

```

```

# -----
# 6. TARGET CODE GENERATION (emit SVG)
# -----

```

```

def generate_svg(ir, width=400, height=400, margin=200):
    lines = []
    for instr in ir:
        if instr[0] == "LINE":
            _, x1, y1, x2, y2, color = instr
            lines.append(
                f'<line x1="{x1+margin:.2f}" y1="{y1+margin:.2f}" '
                f'x2="{x2+margin:.2f}" y2="{y2+margin:.2f}" '
                f'stroke="{color}" stroke-width="2" />'
            )
    body = "\n ".join(lines)
    svg = f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox="0 0 {width} {height}">
    <rect width="100%" height="100%" fill="white" />
    {body}
    </svg>'
    return svg

```

```

# -----
# DRIVER
# -----

```

```

def compile_source(source, out_path="output.svg", show_ir=False):
    tokens = tokenize(source)
    ast = Parser(tokens).parse_program()
    check_semantics(ast)
    ir = generate_ir(ast)
    if show_ir:
        print(f"Unoptimized IR: {len(ir)} instructions")
        for instr in ir:
            print(" ", instr)
    opt_ir = optimize_ir(ir)

```

```
if show_ir:
    print(f"Optimized IR: {len(opt_ir)} instructions")
    for instr in opt_ir:
        print(" ", instr)
svg = generate_svg(opt_ir)
with open(out_path, "w") as f:
    f.write(svg)
return out_path, len(ir), len(opt_ir)
```

```
if __name__ == "__main__":
    src_path = sys.argv[1] if len(sys.argv) > 1 else "examples/square.sketch"
    with open(src_path) as f:
        source = f.read()
    try:
        out, before, after = compile_source(source, "output.svg", show_ir=True)
        print(f"\nCompiled '{src_path}' -> {out}")
        print(f"Optimizer reduced {before} IR instructions to {after}.")
    except (LexError, ParseError, SemanticError) as e:
        print(f"Compilation failed: {e}")
    sys.exit(1)
```